

大型语言模型（LLM）、检索增强生成（RAG）和智能体（Agent）

随着人工智能技术的飞速发展，大型语言模型（LLM）、检索增强生成（RAG）和智能体（Agent）已经成为推动该领域进步的关键技术，这些技术不仅改变了我们与机器的交互方式，而且为各种应用和服务的开发提供了前所未有的可能性。正确理解这三者的概念及其之间的关系是做好面向AI编程开发的基础：

	大模型（LLM）	检索增强生成（RAG）	智能体（Agent）
定义	大型语言模型（LLM），如GPT系列、BERT等，是利用大量文本数据训练的模型，能够生成连贯的文本、理解语言、回答问题等。	检索增强生成技术结合了传统的信息检索技术和最新的生成式模型。它先从一个大型的知识库中检索出与查询最相关的信息，然后基于这些信息生成回答。	智能体是指具有一定智能的程序或设备，能够感知环境并根据感知结果做出响应或决策的实体。它们可以是简单的软件程序或复杂的机器人。
作用	LLM作为基础技术，提供了强大的语言理解和生成能力，是构建复杂人工智能系统的基石。	RAG可以视为在LLM基础上的扩展或应用，利用LLM的生成能力和外部知识库的丰富信息来提供更准确、信息丰富的输出。	智能体可以利用LLM进行自然语言处理，通过RAG技术获得和利用知识，以在更广泛的环境中做出决策和执行任务。它们通常位于应用层级，是对LLM和RAG技术在特定环境下的集成和应用。

从层级关系上看，大模型（LLM）提供了基础的语言理解和生成能力。在此基础上，检索增强生成（RAG）技术利用这种能力结合特定的知识库来生成更为准确和相关的输出。智能体（Agent）则在更高层次上使用LLM和RAG，结合自身的感知和决策能力，在各种环境中执行具体的任务。

因此，可以理解为LLM是基础，RAG是在LLM基础上的进一步应用，而智能体则是综合运用LLM和RAG以及其他技术，在更复杂环境中进行交互和任务执行的实体。这种关系体现了从基础技术到应用技术再到实际应用的逐级深入。

随着技术的快速进步，如何更高效地利用这些大模型（LLM）来解决具体问题？如何通过检索增强生成（RAG）技术提高信息的准确性和相关性？以及如何设计能够有效集成LLM、RAG和其他AI技术的智能体？这些问题的解决，不仅需要深入理解这些技术的工作原理和应用场景，还需要探索它们之间的相互作用和集成方法。

大模型（LLM）的概念与工程化实践

大型语言模型（LLM），如OpenAI的GPT系列，是一种基于深度学习的自然语言处理技术。它们能够理解、生成、翻译文本，完成问答任务，甚至编写代码。这些模型通过在大规模文本数据上的预训练，学会了语言的复杂结构和丰富的知识，使其能够在没有明确指示的情况下执行各种语言任务。GPT系列模型基于变换器（Transformer）架构，这是一种高效的深度学习模型结构，特别适合处理序列数据，如文本。变换器利用自注意力（self-attention）机制，能够捕捉文本中长距离的依赖关系，这对于理解和生成自然语言（NLG）至关重要。

目前，OpenAI最新版本的LLM工程化应用是以GPT-4为基础的，针对普通用户有3个版本，分别是免费版本（只能使用GPT-3.5）、Plus版本以及团队版本（Plus的功能加上团队协作工作管理）。每个月支付20美元（不含税）即可使用Plus版本，即ChatGPTPlus，它的主要功能有：

Chat（对话）

与“OpenAI最强大的模型GPT-4”进行对话，不止是文本的交互生成，还可以同时进行基于DALL-E的图文交互生成，以及从互联网实时获取最新知识进行辅助分析和生成。如下图：

GPTs（插件）

如果你想将自己独有的指令、知识库或任何能力的API服务，同预训练的GPT-4 LLM结合在一起，创建一个“自定义模型”，那么，可以使用“GPTs”插件功能在Open AI的Web应用上快速构建出来。GPTs的推出体现了OpenAI与众不同的工程化创新能力，其交互设计理念值得我们借鉴。使用它的步骤可以参考如下这个例子：

1. 告诉 GPT Builder向导（实际上这也是一个官方的GPTs）你要做什么，它会提示你可以这样说：“制作一个帮助生成新产品视觉效果创意人”或“制作一个帮助我格式化代码的软件工程师”。如下图：

2. 输入“创建一个物流系统的技术支持工程师，帮助商家解答系统问题和处理异常订单”，接下来GPT Builder会和你做一些简单的对话，比如征求你对于命名、Logo的建议等等，如下图：

3. 仅需要2轮简短对话，一个名为“小狗物流平台技术支持”的GPTs被初步创建出来了。生成的“Instrucitons”部分可以视为GTP的System Prompt（系统提示），需要特别注意按照这4个维度修正Instrucitons，直到其准确符合你的意图：1) 定位，希望GPTs执行什么类型的任务；2) 上下文，给GPTs提供一些额外的信息，比如垂直领域的常识，从而引导其给出更好的回答；3) 输入数据，“限定”GPTs引导用户提出的问题，确保不偏离主题；4) 输出数据，“限定”GPTs给出指定格式和范围的输出，确保不输出无关的内容。如下图：

4. 重点来了，在这里可以使用“Upload files”功能上传你自己的“知识库”文件给到大模型推理，文件可以是文档、表格、图片等多种格式，这可以理解是一种对LLM的“静态”增强。如下图：

5. 更重要的是，可以通过添加“Actions”的方式，接入你的API服务给到大模型调用，API通过遵循OpenAPI3的规范进行自描述。大模型可以根据API的功能描述以及输入输出定义，结合用户会话上下文进行智能的调用，获取你的私域数据进行推理，这可以理解是一种对LLM的“动态”增强。如下图：

6. 最后，你可以把你精心“调教”出来的“自定义模型”分享给任何人或者发布到OpenAI的GPTs商店，如下图：

GPTs商店自2024 年初上线以来3个月时间，已经有超过 300 万个自定义的 ChatGPT发布。商店的功能包括2个排行榜，分别是“顶级推荐”和“流行趋势”；具体的分类有 DALL-E 图像创作、写作、效率、研究和分析、编程、教育以及生活方式共7项，并且将由ChatGPT官方创作的自定义模型进行单独分类展示。例如在研究和分析（Research & Analysis）类排名第二的“Scholar GPT”能够利用Google Scholar、PubMed、JSTOR、Arxiv等学术库的2亿+资源和内置的批判性阅读技能，助力你提高研究水平，可谓是撰写论文的神器；在效率（Productivity）类排名第一Canva能够轻松帮助你设计演示文稿、徽标、图文混排等多种内容，并且支持你直接在其提供的Web应用上对AI生成的源文件进行编辑调整，直到达成满意的效果。目前，已经有创作者通过GPTs商店独特的AI生态，实现了自己的商业模式，值得我们学习借鉴。

API（开放接口）

如果你不想依赖于OpenAI的生态平台实现自己的AI应用和商业模式，但又想借助其提供的ChatGPT等基础能力，那么，通过调用其对外开放的API接口一直是最好的选择。因此，OpenAI在推出GPTs的同时，也快速的上线了“Assistant API”的Beta版本，在这个版本中，你可以实现GPTs中提到的所有“增强”大模型的能力，并通过API的方式将其对外发布，供第三方应用调用，并且支持GPT-4模型（当然调用价格也是不菲），如下图：

同时，你仍然可以通过传统的“fine-tuning model”API定制自己的微调大模型，这种方式主要是通过你上传格式化的“问-答”型的训练数据文件来实现对LLM的“增强”。相对于最新推出的“Assistant API”，感觉这种方式在工程化的显得不够灵活和直接，不是很“智能”，目前“fine-tuning model”最高也只能支持GPT-3.5系列模型。如下图：

检索增强生成（RAG）技术概述和应用

通过上一章的介绍，你可以发现OpenAI已经大规模使用工程化的技术使用户能够基于自己的知识库对其GPT系列大模型进行“增强”，从而实现更加垂直化、个性化的能力。那么，如果你基于成本或安全的考虑，想在私域进行自有知识库的“增强”，甚至切换成其它的大模型来使用这个“增强”，就不得不考虑自行开发实现了，这时候就需要了解检索增强生成（RAG）概念和向量数据库技术的应用。

什么是检索增强生成

检索增强生成（RAG）技术人工智能的应用方法，它通过结合传统的信息检索技术与最新的生成式深度学习模型，来提升信息的准确性和相关性。RAG理论来自于2020年Facebook的论文 Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks（知识密集型自然语言处理任务的检索增强生成，原文：<https://arxiv.org/abs/2005.11401>）。RAG的核心思想是在大模型生成回答之前，先从一个知识库中检索出与查询最相关的信息，然后基于这些信息生成准确的回答。

一般来说RAG的工作原理是：首先利用一个检索组件在知识库（这个过程一般使用向量数据库实现）中查找与用户查询相关的文档或数据，这一步骤确保了生成过程基于的是与查询高度相关的信息。随后，这些检索到的信息被送入一个生成模型（如GPT系列大模型），该模型结合检索到的信息和原始查询生成详尽的回答或内容。其核心流程如下图（参考：<https://aws.amazon.com/cn/what-is/retrieval-augmented-generation>）：

RAG典型的应用场景一般在问答系统、内容推荐、数据分析领域。其优势主要在于能够结合检索结果生成回答，提高了只依赖大模型回答的准确度、实时性和信息的丰富性。

为什么是向量数据库

上面我们提到RAG技术一般会使用向量数据库做为“知识库”来支撑用户存储和检索自己的文档或数据。关于向量数据库的理论和概念最近随着AIGC的火热被谈论的很多了，这里有2篇文章能让你快速的了解它：

1.介绍向量数据库的数学理论：相似性度量——余弦相似度和点积，曼哈顿距离（L1）和欧几里得距离（L2），<https://cloud.tencent.com/developer/article/2336891>

2.介绍向量数据库的概念、原理、算法以及选型：
<https://guangzhengli.com/blog/zh/vector-database>

对于RAG（检索增强生成）技术方案来说，为什么使用向量数据库，主要是因为其不仅能提供传统的结构化/非结构化数据库增删改查（CRUD）以及元数据管理的能力，还在处理高维数据，特别是处理深度学习模型生成的向量数据方面具有特殊的优势，具体表现在：

1.维度。深度学习模型生成的文本、图像或语音向量通常位于高维空间中。传统的关系型数据库并不擅长处理这类高维数据，因为它们主要是为处理结构化数据（如表格数据）而设计的。相比之下，向量数据库天生就是为了存储和管理高维空间数据而构建，能够有效地处理和存储这类数据。

2.速度。向量数据库专门设计用于存储高维向量，并支持快速的相似性搜索。在RAG技术中，需要从大量数据中检索与查询最相关的信息，这通常涉及到计算查询向量与数据库中所有向量之间的相似度。向量数据库通过优化的索引结构和近似最近邻（ANN）搜索算法，能够高效地完成这一任务，显著提高检索速度。

3.推理。向量数据库支持基于向量相似度的复杂查询，这对于RAG技术中的自然语言查询处理至关重要。它们可以根据查询的语义内容相关性而非仅仅是关键字匹配来检索信息，这使得向量数据库具有“推理”的能力，而非只是“查询”。

向量数据库一般基于嵌入模型（Embedding Models）将文本向量化，从而来完成推理。前面提到Google发布的BERT模型和OpenAI发布的GPT模型都能提供嵌入（Embedding）计算的能力，但一般BERT系列模型相对于GPT系列模型会“小”很多，这体现在参数数量和磁盘占用上，可以说是“小模型”和“大模型”之分，在做向量计算时该如何选择呢？简单的说它们的相同点都是基于深度学习“将数据转换为高维向量表示”。不同点在于小模型侧重于数据的有效关联判定和简单逻辑推理，而大模型则侧重于深入理解和生成文本等更复杂的任务，具体如下：

	小模型	大模型
设计目标和用途	通常设计为特定任务的一部分，比如将单词、句子或文档转换为向量形式，这些向量随后用于各种任务（如聚类、相似度搜索等）。	为理解文本上下文并生成文本而设计的。可以直接用于生成文本、问答、提取摘要等任务。
模型规模和复杂性	往往相对简单，参数量少，专注于有效地将数据转换为嵌入向量。一般模型主体占用数百MB磁盘空间。	拥有极大的参数量（从几十亿到几百亿不等），设计更为复杂，可以捕获数据中的细微语义和结构。占用TB级磁盘空间（AI估算）。
训练数据和过程	训练通常基于特定任务的数据集，目标是学习良好的数据表示。	通过在庞大的数据集上进行预训练，学习语言的广泛特征和模式，然后可以在特定任务上进行微调（fine-tuning）以提高性能。
在向量数据库中的应用	产生的向量直接用于向量数据库中，以支持快速的相似性检索和推理。	产生的向量可以用于向量数据库。但通常更注重捕捉丰富的语义信息，在需要深度理解的应用场景中作用更大。

下一节的例子会展示以上区别。

目前，在市场上可供选择的向量数据库产品越来越多了，其中Faiss（Facebook AI Similarity Search）、Milvus等产品已经可以用于企业级生产。

Chroma是2023年中旬发布的一个面向AI应用的开源向量数据库，简单、轻量、易用，是专门为自然语言处理（NLP）、图像分类、构建推荐系统和聊天机器人等领域的应用而设计的，非常适合用来快速构建和探索RAG应用。

举个例子

下面用实际Python代码展示一个基于Chroma向量数据库实现RAG关键步骤“文本推理”（对应3.1节示意图环节②③）的例子，分别使用“小模型”和“大模型”对中文文本进行向量化处理，然后针对三个问题进行推理，比较这两种不同模型得到的结果：

1. 创建chroma数据库实例并启动它。当然，在此之前你可能需要用一行代码先安装它“pip install chromadb”，更多的资料可以参考官方文档：

<https://docs.trychroma.com/getting-started>

```
import chromadb

basePath = "/dev/chromadbDemo/"
chroma_client = chromadb.PersistentClient(path=basePath + "chromadata")
print("数据库已启动：" + str(chroma_client))
```

2. 从磁盘上加载4段长文本以及录入4段短文本，用来构建你自己的“知识库”。

```
# -----准备数据
```

```
-----# 红楼梦（千字概述，正常风格）
```

```
file_path_hlm = basePath + "book_HLM.txt"# 金瓶梅（千字概述，正常风格）
file_path_jpm = basePath + "book_JPM.txt"# 水浒传（千字概述，无厘头风格）
file_path_shz = basePath + "book_SHZ.txt"# 指环王（千字概述，莎士比亚风格）
file_path_zhw = basePath + "book_ZHW.txt"docs = [ open(file_path_hlm, "r",
encoding="utf-8").read(), open(file_path_jpm, "r", encoding="utf-8").read(),
open(file_path_shz, "r", encoding="utf-8").read(), open(file_path_zhw, "r",
encoding="utf-8").read(), "不可以，早晨喝牛奶不科学", "吃了海鲜后是不能再喝牛奶的，因为牛奶中含得有维生素C，如果海鲜喝牛奶一起服用会对人体造成一定的伤害",
"吃海鲜是不可以吃柠檬的因为其中的维生素C会和海鲜中的矿物质形成砷", "吃海鲜是不能同时喝牛奶吃水果，这个至少间隔6小时以上才可以"],metas = [ {"source":
file_path_hlm, "uris": file_path_hlm, "author": "曹雪芹"}, {"source": file_path_jpm,
"uris": file_path_jpm, "author": "兰陵笑笑生"}, {"source": file_path_shz, "uris":
file_path_shz, "author": "施耐庵"}, {"source": file_path_zhw, "uris": file_path_zhw,
"author": "托尔金"}, {"source": "my_source1"}, {"source": "my_source2"},
{"source": "my_source3"}, {"source": "my_source4"}],ids = ["id-hlm", "id-jpm",
"id-shz", "id-zhw", "id1", "id2", "id3", "id4"]
```

3. 定义处理数据的4个函数，分别是文本转向量的函数、插入数据表的函数以及2种不同模型创建数据集（可以理解为“数据库表”）的函数。

定义处理数据的函数

用于将文本输入转换为Bert嵌入向量，默认使用 bert-base-chinese 模型和分词器处理文本。

```
def bert_embedding(text, modelName="bert-base-chinese"):  
    from transformers import BertModel, BertTokenizer
```

```

tokenizer = BertTokenizer.from_pretrained(modelName)
model = BertModel.from_pretrained(modelName)
inputs = tokenizer(text, return_tensors="pt", padding=True, truncation=True, max_length=512)
outputs = model(**inputs)
embeddings = outputs.last_hidden_state[:, 0, :].detach().numpy()
return embeddings# 插入数据def setData(collection, embedding=None):
if embedding is None:
    collection.upsert(documents=docs,
                      metadatas=metas,
                      ids=ids,
                      )
else:
    collection.upsert(documents=docs,
                      embeddings=embeddings,
                      metadatas=metas,
                      ids=ids,
                      )
return collection# 使用指定的嵌入模型建数据集，不指定则默认为：Sentence Transformers all-MiniLM-L6-v2
def getDefaultEmbeddingCollection(embeddingModelName=""):
    collection = chroma_client.get_or_create_collection(name="collection_default")
    if embeddingModelName is None or not embeddingModelName:
        # 默认的向量模型
        setData(collection)
    else:
        embedding = bert_embedding(docs, embeddingModelName)
        collection = setData(collection, embedding)
        collection.name = "collection_" + embeddingModelName + "用OpenAI的collection# 使用embedding-ada-002模型建数据集def getOpenAIEndingCollection():
import chromadb.utils.embedding_functions as embedding_functions

    openai_ef = embedding_functions.OpenAIEndingFunction(api_key="[填入你的 OpenAI API Key]",
                                                         model_name="text-embedding-ada-002",
                                                         )
    collection = chroma_client.get_or_create_collection(name="collection_text-embedding-ada-002", embedding_function=openai_ef
    )
    setData(collection)
    return collection

```

4. 在这里定义3个问题，用来测试基于不同模型数据集的推理能力。同时定义一个函数，打印推理结果。

```

collections = chroma_client.list_collections()print("现有数据集：" + str(collections))# 三个问题，用来测试不同数据集和向量模型的推理能力q1 = "我
想了解中国四大名著"q2 = "关于宋朝发生的故事"q3 = "吃完海鲜可以喝牛奶
吗?"def testModel(collection, q, rtNum, embeddingModelName=None):
    if embeddingModelName is None:
        results = collection.query(query="text-embedding-ada-002", results=results) + "\n"
        else:

        results = collection.query(query_embeddings=bert_embedding(q, embeddingModelName), results=results)
        print(q + " - 查询结果：" + str(results) + "\n")

```

有了以上的准备，就可以开始测试了。

首先我们使用“bert-base-chinese”这样的“小模型”对问题进行推理测试，这是Google基于BERT架构（Bidirectional Encoder Representations from Transformers）推出的中文预训练模型，能够理解中文语境和语义，模型本身约400+MB（参考：

<https://huggingface.co/google-bert/bert-base-chinese>），执行Python可以自动下载到本地。测试代码如下：

```
modelName = "bert-base-chinese"
collection = getDefaultEmbeddingCollection(modelName)
print("当前collection：" + str(collection) + "\n")
testModel(collection, q1, 2, modelName) #问题1返回2笔推理结果
testModel(collection, q2, 3, modelName) #问题2返回3笔推理结果
testModel(collection, q3, 5, modelName) #问题3返回5笔推理结果
```

执行上述Python代码，截取控制台打印的相关输出如下：

```
当前collection：name='collection_bert-base-chinese' id=UUID('d0fe761d-3e64-4b89-ab8a-59a7253d44a7') metadata=None tenant='default_tenant' database='default_database'
想了解中国四大名著 – 查询结果：
{'ids': [['id1', 'id4']], 'distances': [[202.48262633262166, 266.160556742396]], 'metadata': [{'source': '/dev/chromadbDemo/book_HLM.txt', 'uris': '/dev/chromadbDemo/book_HLM.txt'}], 'uris': None, 'data': None}
可以，早晨喝牛奶不科学，'吃海鲜是不能同时喝牛奶吃水果，这个至少间隔6小时以上才可以']], 'uris': None, 'data': None}
关于宋朝发生的故事 – 查询结果：
{'ids': [['id1', 'id3', 'id4']], 'distances': [[253.08461381250856, 300.2129506027819, 334.160556742396]], 'metadata': [{'source': '/dev/chromadbDemo/book_HLM.txt', 'uris': '/dev/chromadbDemo/book_HLM.txt'}], 'uris': None, 'data': None}
可以，早晨喝牛奶不科学，'吃海鲜是不可以吃柠檬的因为其中的维生素C会和海鲜中的矿物质形成砷，'吃海鲜是不能同时喝牛奶吃水果，这个至少间隔6小时以上才可以']], 'uris': None, 'data': None}
吃完海鲜可以喝牛奶吗？ – 查询结果：
{'ids': [['id1', 'id2', 'id4', 'id3', 'id-hlm']], 'distances': [[173.57739555949934, 201.32507459764457, 202.22220711154088, 261.7239443921094, 452.04586252776966]], 'metadata': [{'source': 'my_source1'}, {'source': 'my_source2'}], 'uris': None, 'data': None}
雪芹，'source': '/dev/chromadbDemo/book_HLM.txt', 'uris': '/dev/chromadbDemo/book_HLM.txt'}], 'uris': None, 'data': None}
可以，早晨喝牛奶不科学，'吃了海鲜后是不能再喝牛奶的，因为牛奶中含得有维生素C，如果海鲜喝牛奶一起服用会对人体造成一定的伤害，'吃海鲜是不能同时喝牛奶吃水果，这个至少间隔6小时以上才可以，'吃海鲜是不可以吃柠檬的因为其中的维生素C会和海鲜中的矿物质形成砷，'《红楼梦》是清代曹雪芹创作的一部长篇小说，被誉为中国古代四大名著之一。该作品通过贾、王、史、薛四大家族的兴衰史，细腻地描绘了封建王朝末期的社会生活，深刻揭示了封建社会的腐朽与衰败.....此处省略1000字']], 'uris': None, 'data': None}
```

解读一下关键信息：ids字段是返回的结果标识；distances字段是向量距离，意思是问题和结果的相关性，距离越短表示越相关；documents字段则是返回的具体结果数据。可以发现：

- 1.“bert-base-chinese”模型的distances一般算出来是百位数；
- 2.“小模型”是可以基于简短文本数据进行一些简单推理的，例如“bert-base-chinese”模型对问题3“吃完海鲜可以喝牛奶吗？”的推理结果“基本”合格；
- 3.“小模型”基于较长文本数据的推理很“随机”，效果很差，例如“bert-base-chinese”模型对于问题1“中国四大名著”和问题2“宋朝的故事”的问题就完全无法理解，尽管在我提供的长文本数据里明显含有这2个问题的关键词。

然后我们使用“OpenAI text-embedding-ada-002”这样的“大模型”对问题进行推理测试。测试代码如下：

```
collection = getOpenAIEmbeddingCollection()print("当前  
collection：" + str(collection) + "\n")testModel(collection, q1, 2) #问题1返回2笔推理结果  
testModel(collection, q2, 3) #问题2返回3笔推理结果testModel(collection, q3, 5) #问题3返回5笔推理结果
```

执行上述代码，得到如下关键输出：

```
当前collection：name='collection_text-embedding-  
ada-002' id='bd211d(ec450ccf-8359-4bfb-f4bde881cb06)' metadata=None tenant='default_tenant' database='default_database'我  
想了解中国四大名著 – 查询结果：{'ids': [['id-hlm', 'id-  
jpm']], 'distances': [[ 0.3868423766235018267]], 'metadatas': [[{'author': '曹雪  
芹', 'source': '/dev/chromadbDemo/book_HLM.txt', 'uris': '/dev/chromadbDemo/book_1  
陵笑笑  
生', 'source': '/dev/chromadbDemo/book_JPM.txt', 'uris': '/dev/chromadbDemo/  
book_JPM.txt'}]]关于《红楼梦》是清代曹雪芹创作的长篇小说，被誉为中国四大名著之一。该作品通  
过贾、王、史、薛四大家族的兴衰史，细腻地描绘了封建王朝末期的社会生活，深刻揭示了  
封建社会的腐朽与衰败。小说以贾宝玉和林黛玉的爱情悲剧为主线，通过丰富的人物群像和  
错综复杂的情节展现了一个广阔的社会生活画卷.....此处省略1000字，' 《金瓶梅》是中  
国文学史上的一部重要小说，被认为是明代中期的作品，作者一般被认为是兰陵笑笑生。这  
部小说以宋代开封为背景，详细描绘了主人公西门庆与他的家人、情人、朋友之间的复杂关  
系，以及由此引发的一系列社会和家庭冲突.....此处省略1000  
字']], 'uris': None, 'data': None}关于宋朝发生的故事 – 查询结果：{'ids': [['id-shz', 'id-  
jpm', 'id-zhw']], 'distances': [[ 0.322985944455922, 0.3312445684997755,  
0.33733609769548206]], 'metadatas': [[{'author': '施耐  
庵', 'source': '/dev/chromadbDemo/book_SHZ.txt', 'uris': '/dev/chromadbDemo/book_S
```

陵笑笑

```
生', 'source': '/dev/chromadbDemo/book_JPM.txt', 'uris': '/dev/chromadbDemo/book_J
```

尔

金’, ’source’: ’/dev/chromadbDemo/book_ZHW.txt’, ’uris’: ’/dev/chromadbDemo/book_ZHW.txt’, ’distance’: 0.0}], ’uris’: None, ’data’: None}

还有W.txt本文人眼花缭乱的古典名著，故事内容丰富得可以用来炒一大锅剧情泡面。整个故事发生在北宋时期，可以想象成一个古代的超级英雄联盟，但这些英雄不穿紧身衣，而是穿着古代汉服，横扫江湖，打击不公……此处省略1000字’， ’《金瓶梅》是中国文学史上的一部重要小说，被认为是明代中期的作品，作者一般被认为是兰陵笑笑生。这部小说以宋代开封为背景，详细描绘了主人公西门庆与他的家人、情人、朋友之间的复杂关系，以及由此引发的一系列社会和家庭冲突……此处省略1000字’， ’在中世纪幻想的土地，被称为中土的地方，诞生了一部伟大的故事——《指环王》。这部史诗般的作品，如同莎士比亚之笔下的戏剧，充满了权力的争斗、勇气的考验、忠诚与背叛的较量，以及对自由与爱的无尽追求。

噢，听吧，那遥远的号角在召唤，就如同命运之神在低语，引领我们走向那个被称为“魔戒”的强大而又可怕的物品……此处省略1000字’]], ’uris’: None, ’data’: None}

吃完海鲜可以喝牛奶吗？ – 查询结果：{‘ids’: [[‘id2’, ’id4’, ’id3’, ’id1’, ’id-shz’]], ’distances’: [[0.18699816057051363, 0.2437766582633824, 0.3233349839279665, 0.33243019058071627, 0.5406020260719162]], ’metadatas’: [[{‘source’: ’my_source2’}, {‘source’: ’my_source4

耐

庵’, ‘source’: ‘/dev/chromadbDemo/book_SHZ.txt’, ‘uris’: ‘/dev/chromadbDemo/book_SHZ.txt’], ‘data’: None}

了海鲜后是不能再喝牛奶的’, ‘source’: ‘/dev/chromadbDemo/book_SHZ.txt’, ‘uris’: ‘/dev/chromadbDemo/book_SHZ.txt’], ‘data’: None}

对人体造成一定的伤害’, ‘吃海鲜是不能同时喝牛奶吃水果, 这个至少间隔6小时以上才可以’, ‘吃海鲜是不可以吃柠檬的因为其中的维生素C会和海鲜中的矿物质形成砷’, ‘不可以, 早晨喝牛奶不科学’, ‘水浒传, 一本让人眼花缭乱的古典名著, 故事内容丰富得可以用来炒一大锅剧情泡面。整个故事发生在北宋时期, 可以想象成一个古代的超级英雄联盟, 但这些英雄不穿紧身衣, 而是穿着古代汉服, 横扫江湖, 打击不公……此处省略1000字’], ‘uris’: None, ‘data’: None}

对比第一次小模型测试的结果，我们可以明显感觉到：

1.“OpenAI text-embedding-ada-002”模型的distances算出来都是小于1，但含义仍然是“离越小越相关”；

2.“大模型”基于简短文本数据的推理相当精确，针对问题3“吃完海鲜可以喝牛奶吗？”的推理结果堪称完美，相对于之前的“小模型”结果，“大模型”能准确的把“海鲜与柠檬”、“早晨喝牛奶”这类相关性较差数据的向量距离依次排开，并且能把“水浒传”这类相关性极差数据的向量距离明显拉开。

3.“大模型”基于较长文本数据的推理在这个测试中都在首位命中了事实上最相关的结果，例如它能在问题1“我想了解中国四大名著”的推理中把描述红楼梦的数据排在第1位以及问题2“关于宋朝发生的故事”的推理中把描述水浒传的数据排在第1位。但也都有不足，例如它在问题1“我想了解中国四大名著”的推理结果中把描述金瓶梅的数据排在第2位，按照常识应该水浒传才是四大名著之一；在问题2“关于宋朝发生的故事”的推理结果中把描述指环王的数据排在第3位，而按照常识红楼梦似乎和中国、宋朝的相关性比指环王会更高一些。

上述的例子使用Python代码编写，当然也可以使用Java实现，Chroma也有相关的Java SDK可以使用。如果说Java是企业应用时代的原生语言，那么Python就是AI时代的原生语言，大多数AI项目的官方支持语言都是Python，并且相对于Java来说，Python的学习和应用更加简便，建议直接用起来。

基于上述简单的测试证明我们可以将类似OpenAI “text-embedding-ada-002”这样的大模型应用到实际的RAG生产中，事实上目前京东很多AI客户服务使用的向量嵌入模型正是OpenAI “text-embedding-ada-002”。鉴于text-embedding-ada-002这个模型是GTP3时代的产品，相信未来OpenAI推出基于GTP4的嵌入模型一定会更加强大精准。

而小模型对于本地化训练、垂直领域RAG应用，特别是学习研究AI技术是更方便的。如果你不想编写代码，有一些网站也能提供良好的沙箱，供你学习、调试小型嵌入模型，这些模型不仅限于文本，还包括图像、语音，甚至视频，国外的有“抱脸”<https://huggingface.co>，国内的有“魔搭”<https://modelscope.cn>，都值得一试。

智能体（Agent）的概念、应用和集成

智能体的概念和开发思想

上面我们提到可以利用RAG技术结合自有知识库对大模型进行增强，从而获得更准确、实时、丰富的垂直内容或个性化结果。但这仍然没有跳出内容生成（AIGC）的范畴，如果你需要人工智能像一个“以终为始”的高效率员工一样自主选取各种工具、和各种不同系统沟通协同工作，直到交付最终结果，那么就需要了解“智能体”这个方案了。

智能体（Agent）技术是人工智能应用的一个核心概念，是指可以自主执行任务、作出决策，并在一定程度上模拟人类或其他智能实体行为的计算机程序或机器。目前智能体在仿真、游戏、客户服务以及自动化控制等多个领域和应用中展示了巨大的潜力，从简单的自动化脚本到复杂的决策支持系统，智能体在软件和硬件系统中通过扮演感知者、执行者、决策者或学习者等多种角色来完成任务。

关于智能体的理论依据可以参考发表于2023年的论文 Plan-and-Solve Prompting: Improving Zero-Shot Chain-of-Thought Reasoning by Large Language Models (计划与解决提示：通过大型语言模型改进零点思维链推理，原文：<https://arxiv.org/abs/2305.04091>)，文章主要提出了一种改进大型语言模型在零次学习环境下执行多步骤推理任务的方法，作者通过三个步骤来论证：第一步通过几个手工制作的逐步推理示例 (few-shot chain-of-thought prompting)，证明大模型能够明确地生成推理步骤并提高其推理任务的准确性；第二步为了消除手动制作示例的工作量，作者提出了零样本思维链方法 (Zero-shot Chain-of-Thought)，该方法将目标问题声明与“让我们一步步来思考”的引导过程作为输入提示给大模型；第三步，为了解决零样本思维链方法存在的“漏步骤”错误，作者提出了“先计划再求解 (PS, Plan-and-Solve)”的提示策略。

文章提出的“先计划再求解”原理是一些智能体产品设计开发的核心思想，其遵循以下几个步骤：

- 1.明确问题：首先，需要明确问题的核心内容和目标。即使问题未知，也可以通过一些通用的启动语句引导模型聚焦问题，通用的提示可以是：“让我们首先理解问题的核心并制定一个解决方案的计划。”
- 2.划分子任务：对于大部分复杂问题，可以将其分解为更小的、更易于管理和解决的子任务。通用的提示可以是：“接下来，根据这个计划分解问题为子任务。”
- 3.制定具体计划：基于子任务，进一步细化出解决每个子任务的具体计划。可以使用类似于：“针对每个子任务，让我们制定一个具体的解决方案或步骤。”的提示。
- 4.执行并验证：鼓励模型按照计划执行，并在执行过程中检验每一步的正确性。可以用：在现按照我们的解决方案计划逐步执行，并检查每一步的结果是否符合任务的预期提示来指导模型。
- 5.适时调整：如果在执行过程中发现问题或者结果与预期不符，需要准备好调整计划的提示，例如补充类似的提示：“如果我们发现任何步骤的结果不如预期，让我们回顾并调整我们的计划。”
- 6.总结答案：最后，鼓励模型汇总执行计划后的结果，给出最终的答案或解决方案，提示可以是：“最后，综合我们所有子任务解决方案执行的结果，总结出问题答案或最终解决方案。”

举个例子

在2022年大模型浪潮爆发后不久，开源的智能体项目也跟着大量的涌现。从最早AutoGPT推出时的轰动，到目前MetaGPT的逐步应用，我通过自己的跟踪测试能深切的体会到这项技术从“一言难尽”到日趋成熟，在工业生产中的使用指日可期。

AutoGPT和MetaGPT都是非常有趣的开源智能体产品，值得学习和借鉴。AutoGPT的运

AUTOGPT 和 MetaGPT 都是非常有趣的开源智能体产品，值得学习和借鉴。AUTOGPT 的运行可能会麻烦一些，建议在 Docker 环境下跑效果会更好，参考官方文档：

<https://docs.agpt.co>；MetaGPT 可以比较方便的在 Python 环境下运行，而且有中文官方文档：https://docs.deepwisdom.ai/main/zh/guide/get_started/introduction.html。

如果你不想编写代码，也快速体验智能体产品，使用 Tavily 会比较方便。Tavily 的口号是“对耗时的研究说再见”——声称可以帮助你快速洞察和全面研究一个课题，从准确收集信息源到整理研究成果，并且将所有工作都集中在一个平台上完成。商业模式是对 API 调用次数及调用其高质量实时知识库增强推理（RAG）付费，值得借鉴。

Tavily 可以提供一個沙箱（API Playground），供用户测试其“针对高质量实时知识优化的大模型增强检索”API，沙箱只开放通用知识库（垂直的高质量实时知识库需要付费才能使用），沙箱的测试结果会返回互联网上的相关知识集。我提出了一个极富挑战性的课题进行测试，效果如下图：

Tavily 还提供了一个研究助手（Research Assistant）的 Web 应用，供用户在线使用智能体产品，需要用户使用自己的 OpenAI API Key 调用 GPT4 服务，支持“深度研究”，也就是要消耗更多的算力和时间（钱）获得更高质量的结果。我提出了一个很直接很实用的课题进行测试，消耗了大约 2 元多人民币的 API 调用费，获得了一份大约 1500 字的报告，过程如下图：

后面我继续针对这个课题测试了几轮，感觉还是有一些正确的分析和建设性的结果，但也存在比较多的“幻觉”，需要用户自行甄别，不完美，有提升的空间。有条件的话可以试用一下：<https://app.tavily.com/chat>。

集成大模型、RAG和智能体的方法和场景

通过前面的介绍，我们能够理解大模型、RAG和智能体这些技术和理念的潜力在于相互结合，形成更为强大和灵活的AI系统——即结合大模型的深层次语言理解和生成能力、RAG的垂直和实时的信息检索能力以及智能体的决策和执行能力。

这种集成可以通过多种方式实现，例如，通过中间件来协调不同技术的交互，或者通过在一个统一的框架下直接整合核心技术。目前，后者是比较主流的方式，因为 [LangChain](https://www.langchain.com) (www.langchain.com) 这个开源的AI开发门面和编排框架用来做这件事非常优秀。LangChain提供了一个框架，允许开发者将大模型“转化”为能够执行一系列动作的智能体，智能体利用大模型作为“推理引擎”，来动态的确定要采取哪些操作、使用哪些工具以及按什么顺序执行。LangChain框架主要能够提供Chain和Agent工具来帮助你构建智能体，Chain能够最大程度的方便开发者将不同的操作和处理步骤以链式的方式组合成AI流程；Agent工具由Agent类型、AgentExecutor、Tools（支持的工具列表：<https://python.langchain.com/docs/integrations/tools>）这几个部分构成，帮助开发者将实时信息交互、外部数据获取、三方系统调用等功能集成到“链”中以扩展或改善AI能力。例如：通过Google搜索获取实时信息、从OpenWeatherMap获取天气信息，以及从Wikipedia获取百科知识。可以说，如果开发AI应用，LangChain不可或缺。（参考资料：<https://www.ibm.com/topics/langchain>）

目前，已经有一些比较典型的行业应用方案：

案例1：智能客服系统。在一个集成了大模型、RAG和智能体的智能客服系统中，大模型可用于理解用户的查询和生成自然语言回复，RAG技术可用于从企业的数据库和知识库中检索准确的信息以支持回复，而智能体则负责管理对话流程、处理事务性任务和执行复杂的用户请求。这种集成使得客服系统能够提供更准确、更人性化的服务，同时减轻人工客服的负担。

案例2：个性化教育平台。在个性化教育平台的例子中，大模型可以根据学生的学习进度和偏好生成定制的学习材料和测试，RAG技术可以从广泛的教育资源中检索相关信息以丰富教学内容，而智能体可以根据学生的反馈和学习成效调整教学策略和内容。这种集成不仅能够为学生提供更加个性化的学习体验，还能够帮助教师更好地理解学生的需求和进步。

案例3：复杂决策支持系统。集成大模型、RAG和智能体的复杂决策支持系统能够在金融、医疗和科研等领域提供强大的支持。在这种系统中，大模型用于处理和生成语言信息，RAG技术用于从大量数据中检索相关信息和案例，智能体则负责综合这些信息，形成决策建议。这种集成系统能够处理复杂的查询，提供基于数据的决策支持。

针对供应链物流领域通过集成大模型、RAG和智能体技术，可以从如下几个业务系统探索突破点：

1. 仓储管理（WMS）：结合RAG技术和智能体，系统能够实时从供应商数据库、仓库库存记录和销售数据中检索关键信息，智能调整库存水平，减少库存积压和缺货风险。

2. 运输管理（TMS）：通过分析地理位置数据、运输成本和时间要求，智能体可以规划最优的货物配送路线和调度计划。这一过程会利用大模型来处理复杂的逻辑和约束条件，以确保高效且成本效益的配送。

3. 供应链销售与运营规划（S&OP）：利用大模型处理历史销售数据、市场趋势和用户反

反馈，生成精准的需求预测报告。智能体能够学习并适应市场变化，实时调整预测模型，提高预测的准确性。

未来展望与挑战

对于未来，可以预见，AI的开发和应用会面临诸多的机遇与问题，诸如更加深度的领域知识融合、数据隐私和安全性等等。但最主要的趋势一定是未来的AI系统将具备更强的自适应学习能力并跳出虚拟世界，正如2023年黄仁勋提到人工智能的下一波浪潮是“具身智能

(Embodied AI)”即AI不仅能够理解和处理信息，还能够在物理世界中执行任务和作出反应。因此，智能体 (Agent) 在软件领域会快速的发展，当其足够成熟时，驱动硬件的智能体，即具身智能的规模化应用将是水到渠成的事情。随之而来的，AI伦理问题一定会受到越来越多的挑战：AI应用决策过程的透明度和可解释性、AI决策结果的偏见不公平甚至歧视、人类伦理原则是否得到遵守等，2024年春节期间和生活在欧洲的同学交流得知，目前一些西方国家已经在开展一些“反人工智能 (AntiAI)”的运动和研究，但主流思想并不是反对人工智能的发展，而是建立人工智能治理框架和开发监管技术（例如区分AI生成的内容和人类生成的内容），以确保人工智能的发展优先考虑人类福祉、伦理因素和安全（参考资料：<https://dotcommagazine.com/2023/08/anti-ai-top-ten-things-you-need-to-know>）。

另外AI技术发展与社会发展的和谐相融的问题也非常值得思考，如何避免AI造成的技术性失业这个问题，对我们软件开发者而言显得特别重要。2024年3月在央视《对话》节目上，李彦宏表示，“以后不会存在程序员这种职业了，因为只要会说话，人人都会具备程序员的能力……未来的编程语言只会剩下两种，一种叫做英文，一种叫做中文”。回想20年前的软件开发行业，除了测试，诸如需求分析、画原型、网页开发、桌面开发、后台服务开发，甚至界面设计（美工兼职辅助平面设计）都是由程序员来做，几乎所有当时的“大厂”皆是如此，后来随着技术的演进和社会的发展，不仅编程开发这个行当分成了前端开发、后端开发、客户端开发、大数据开发、算法开发等诸多工种，连“美工”都分化成了视觉、交互、用研等专业领域，那么，未来在AI技术的发展下，这些角色的分工会重新合并，甚至不会存在了么？

可能会出现如下的趋势：

1. 一定会提高效率和创造力。AIGC 可以大大提高设计人员和开发人员的效率，我们很多团队在2023年初就开始使用Midjourney辅助设计或使用JoyCoder和GitHub Copilot辅助编码，效果有目共睹。可以预见，在产品的设计或代码生成等探索性和创造性阶段，通过AI智能体快速生成各种解决方案并将概念转化为可视化表达或代码，会变得越来越便捷并越来越具有实际价值。

2.开发者的角色可能会演变。开发者角色分工的界限会变得越来越模糊，全新的更具协作性和增强型的软件开发生命周期会出现。未来的开发者和设计师会减少用于日常编码或设计，甚至文档撰写的时间，而更多的时间用于明确智能体等AI工具的目标、解释人工智能生成的解决方案，以及将这些解决方案集成到多种不同的系统架构中。开发者的角色可能会演变为承担更多监督和管理的职能，以确保AI生成的设计和代码符合业务目标以及人类价值观。

3.软件开发这个行业会“民主化”。随着 AIGC 工具变得更加易用和强大，我们可能会看到越来越多没有传统设计和编码技能的人也可以发起或参与软件开发。这将使软件开发这个行业参与的业务领域更加多样化，数字化应用的范围更加广泛。

(参考资料：<https://insights.sei.cmu.edu/blog/application-of-large-language-models-llms-in-software-engineering-overblown-hype-or-disruptive-change/>)